알고리즘 중간고사 Summary Sheet

목차

- 1. 용어 사전
- 1주차 알고리즘 입문
- 2주차 복잡도 분석
- 3주차 자료구조와 기본 정렬
- 4주차 분할 정복
- 5주차 그리디
- 6주차 동적 계획법
- 공식 치트시트
- 패러다임 결정 플로우
- 시험 문제 스타일 예시
- 예상 시험문제와 모범 답안 10선
- 자주 하는 실수

---

1. 용어 사전

시험 문제가 영어이므로, 답안에서 한글 용어를 처음 언급할 때 괄호에 영문을 병기한다.

영문	한국어	한 줄 정의
Algorithm	알고리즘	유한 단계로 입력을 받아 출력을 만드는 절차이다.
Correctness	정확성	모든 유효 입력에 대해 올바른 출력을 내는 성질이다.
Finiteness	유한성	유한 시간 안에 반드시 종료되는 성질이다.
Pseudocode	의사코드	자연어와 프로그래밍 구조를 섞어 쓰는 알고리즘 기술 방식이다.
GCD	최대공약수	두 수를 모두 나누는 가장 큰 정수이다.
Big-O	O-표기	점근적 상한을 나타내는 표기이다.
Big-Omega	Ω-표기	점근적 하한을 나타내는 표기이다.
Theta	Θ-표기	꼭 맞는 한계(상한과 하한이 모두 같은 차수)를 나타낸다.
Recurrence Relation	점화식	함수의 값을 더 작은 입력값으로 표현한 식이다.
Master Theorem	마스터 정리	$T(n) = aT(n/b) + f(n)$ 형태의 점화식을 한 번에 풀어 주는 정리이다.
Divide and Conquer	분할 정복	문제를 작은 부분으로 나누고, 각각 풀고, 결과를 결합하는 기법이다.
Greedy	그리디	매 단계에서 지역적으로 최적인 선택을 하는 기법이다.
Dynamic Programming	동적 계획법	중복되는 부분 문제를 한 번만 풀어 저장·재사용하는 기법이다.
Overlapping Subproblems	중복 부분 문제	재귀 호출 구조에서 같은 부분 문제가 여러 번 등장하는 성질이다.
Optimal Substructure	최적 부분 구조	전체 최적해가 부분 문제의 최적해로 구성되는 성질이다.
Memoization	메모이제이션	재귀와 캐시를 결합한 하향식 DP 구현 방식이다.
Tabulation	테이블화	반복문으로 작은 부분 문제부터 채우는 상향식 DP 구현 방식이다.
Stack	스택	마지막에 넣은 원소가 먼저 나오는(LIFO) 자료구조이다.

Queue	큐	먼저 넣은 원소가 먼저 나오는(FIFO) 자료구조이다.
Heap	힙	부모-자식 우선순위 관계를 유지하는 완전 이진 트리이다.
Stable Sort	안정 정렬	같은 키 값을 가진 원소들의 상대 순서를 보존하는 정렬이다.
In-Place	제자리 정렬	추가 메모리를 $O(1)$ 만 사용하는 정렬이다.
Selection Sort	선택 정렬	매 단계마다 최댓값(또는 최솟값)을 찾아 끝으로 옮기는 정렬이다.
Bubble Sort	버블 정렬	인접한 두 원소를 비교·교환하여 큰 값을 끝으로 밀어 올리는 정렬이다.
Insertion Sort	삽입 정렬	이미 정렬된 부분에 새 원소를 올바른 위치에 삽입하는 정렬이다.
Merge Sort	병합 정렬	배열을 절반으로 나누어 각각 정렬한 뒤 병합하는 분할 정복 정렬이다.
Quick Sort	퀵 정렬	피벗을 기준으로 분할한 뒤 양쪽을 재귀적으로 정렬하는 분할 정복 정렬이다.
Heap Sort	힙 정렬	최대 힙을 구성한 뒤 루트를 끝으로 이동해 가며 정렬하는 알고리즘이다.
Counting Sort	계수 정렬	값의 발생 횟수와 누적합으로 정렬하는 비비교 기반 선형 시간 정렬이다.
Radix Sort	기수 정렬	자릿수별로 안정 정렬을 반복하여 전체를 정렬하는 선형 시간 정렬이다.
Binary Search	이진 탐색	정렬된 배열에서 중간값과 비교하여 탐색 범위를 절반씩 줄이는 알고리즘이다.
Partition	파티션	피벗을 기준으로 배열을 두 영역(이하·초과)으로 재배치하는 연산이다.
Pivot	피벗	파티션의 기준이 되는 원소이다.
Selection Problem	선택 문제	정렬되지 않은 배열에서 i 번째로 작은 원소를 찾는 문제이다.
Median of Medians	중앙값의 중앙값	최악의 경우에도 선형 시간을 보장하는 선택 알고리즘의 피벗 전략이다.

Closest Pair	최근접 쌍	평면의 n 개 점 중 거리가 최소인 두 점을 찾는 문제이다.
Greedy-Choice Property	그리디 선택 속성	지역 최적 선택이 언제나 전역 최적해로 이어지는 성질이다.
Fractional Knapsack	분할 가능 배낭	물건을 쪼갤 수 있는 배낭 문제로, 그리디로 최적해를 얻는다.
0/1 Knapsack	0/1 배낭	물건을 쪼갤 수 없는 배낭 문제로, DP로 풀어야 한다.
Activity Selection	활동 선택	한 기계로 수행 가능한 비겹침 활동의 최대 개수를 구하는 문제이다.
Job Scheduling	작업 스케줄링	모든 작업을 수행하기 위한 최소 기계 수를 구하는 문제이다.
Huffman Coding	허프만 코딩	문자 빈도에 따라 최적의 접두 코드를 생성하는 압축 알고리즘이다.
MST	최소 신장 트리	연결 그래프의 모든 정점을 연결하는 가중치 합 최소 트리이다.
Kruskal	크루스칼 알고리즘	간선을 가중치 오름차순으로 보며 사이클이 생기지 않으면 트리에 편입하는 MST 알고리즘이다.
Prim	프림 알고리즘	시작 정점에서 트리를 점진적으로 성장시키는 MST 알고리즘이다.
Dijkstra	다익스트라 알고리즘	음이 아닌 가중치 그래프에서 단일 출발점 최단 경로를 구하는 그리디 알고리즘이다.
Edge Relaxation	간선 완화	경유 정점을 통한 경로가 더 짧으면 거리값을 갱신하는 연산이다.
LCS	최장 공통 부분 수열	두 문자열의 공통 부분 수열 중 가장 긴 것의 길이(또는 문자열)를 구하는 문제이다.
Edit Distance	편집 거리	한 문자열을 다른 문자열로 바꾸는 최소 삽입·삭제·교체 횟수이다.
Floyd-Warshall	플로이드-워셜	모든 정점 쌍의 최단 경로를 구하는 DP 알고리즘이다.
Pseudopolynomial	의사다항 시간	수치 값에는 다항이지만 비트 길이에는 지수인 복잡도이다.

1주차 알고리즘 입문

알고리즘의 다섯 가지 속성

- **정확성(Correctness)** — 모든 유효 입력에 대해 올바른 출력을 내야 한다.
- **실행 가능성(Executability)** — 각 단계가 기계적으로 실행 가능해야 한다.
- **유한성(Finiteness)** — 유한 시간 안에 반드시 종료되어야 한다.
- **효율성(Efficiency)** — 시간과 공간을 적절히 사용해야 한다.
- **명확성(Unambiguity)** — 각 단계가 모호하지 않게 명시되어야 한다.

유클리드 GCD

```
gcd(a, b) = gcd(b, a mod b),  gcd(a, 0) = a
```

설명 — 공약수 집합이 $(a, b) \leftrightarrow (b, a \bmod b)$ 에서 보존되므로 최댓값도 같다. 기저 조건은 $b = 0$ 이며 이때 a 가 정답이다. 복잡도는 최악 $O(\log \min(a, b))$ 이고, 연속한 피보나치 수 쌍에서 최악이 발생한다.

$\log_2 n$ 테마 (절반 분할)

이진 탐색(Binary Search), 가짜 동전(Fake Coin), 독 포도주(Poisoned Wine) 문제는 모두 매 단계 탐색 공간을 절반으로 줄여 $\log n$ 의 성능을 얻는다.

이진 탐색 주의

```
while left <= right:      # 등호 필수
    mid = (left + right) // 2
```

설명 — 정렬된 배열에서 매 단계 중간값과 키를 비교하여 탐색 범위를 절반으로 축소한다. `<=` 에서 등호가 빠지면 `left == right` 인 마지막 한 원소를 검사하지 않아 실패한다. 복잡도는 $O(\log n)$ 이다.

오일러 회로 조건

- **회로 존재 조건** — 모든 정점의 차수가 짝수여야 한다.
- **경로 존재 조건** — 홀수 차수 정점이 정확히 0개 또는 2개여야 한다.
- **주의** — 다리(Bridge, 제거 시 그래프를 분리하는 간선)를 너무 일찍 건너면 반대쪽의 간선을 모

두 순회할 수 없게 된다.

2주차 복잡도 분석

점근 표기법 정의

$$O(g(n)) = f : \exists c > 0, n_0 \geq 0, \forall n \geq n_0, f(n) \leq c \cdot g(n)$$

- **O** — 상한, "최대 이 정도로 증가한다"는 의미이다.
- **Ω** — 하한, "최소 이 정도로 증가한다"는 의미이다.
- **Θ** — 꼭 맞는 한계로, 상한과 하한이 모두 성립한다.

복잡도 계층

```
O(1) < O(log n) < O(√n) < O(n) < O(n log n) < O(n²) < O(n³) <
O(2ⁿ) < O(n!)
```

Big-O 증명 레시피 (다항식)

- 다항식 $f = a_k n^k + \cdots + a_0$ 에 대해 $c = |a_k| + \cdots + |a_0|$, $n_0 = 1$ 로 잡으면 대부분 성립한다.
- **예시** — $2n^2 - 8n + 3 = O(n^2)$ 은 $c = 13$, $n_0 = 1$ 로 증명된다.

점화식 해결 세 가지 방법

1. **반복 대입** — 점화식을 단계별로 전개하여 패턴을 찾는다.
2. **추측 후 귀납 증명** — 해의 형태를 추측하고 귀납법으로 검증하며, 필요하면 $cn\log n - d$ 로 추측을 강화한다.
3. **마스터 정리** — $T(n) = aT(n/b) + f(n)$ 형태의 점화식을 표준적으로 해결한다.

마스터 정리 빠른 참조

Case	조건	결과
1	$f(n) = O(n^{\log_b a - \varepsilon})$	$\Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a})$	$\Theta(n^{\log_b a} \log n)$
3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$ + 정규성 조건	$\Theta(f(n))$

마스터 정리 예시표

점화식	a, b	$n^{\log_b a}$	$f(n)$	Case	결과
$T(n) = 2T(n/2) + n$	2, 2	n	n	2	$\Theta(n\log n)$
$T(n) = T(n/2) + 1$	1, 2	1	1	2	$\Theta(\log n)$
$T(n) = 2T(n/2) + 1$	2, 2	n	1	1	$\Theta(n)$
$T(n) = 4T(n/2) + n$	4, 2	n^2	n	1	$\Theta(n^2)$
$T(n) = 4T(n/2) + n^2$	4, 2	n^2	n^2	2	$\Theta(n^2\log n)$
$T(n) = 7T(n/2) + n^2$ (Strassen)	7, 2	$n^{\log_2 7}$	n^2	1	$\Theta(n^{\log_2 7})$

마스터 정리를 쓸 수 없는 경우

- $T(n) = T(n-1) + n$ 은 반복 대입으로 $\Theta(n^2)$ 이 된다.
- $T(n) = T(n-1) + T(n-2) + 1$ 은 특성 다항식으로 풀면 $\Theta(\phi^n)$ 이 된다.
- $T(n) = T(n/5) + T(7n/10) + n$ 은 귀납법과 기하급수 합으로 $\Theta(n)$ 이 된다.

3주차 자료구조와 기본 정렬

자료구조 요약

구조	접근 방식	주요 연산 복잡도
배열 / 리스트	인덱스 기반	접근 $O(1)$, 중간 삽입·삭제 $O(n)$
스택 (LIFO)	push / pop	모두 $O(1)$
큐 (FIFO)	enqueue / dequeue	모두 $O(1)$
힙	우선순위 기반	insert·extract $O(\log n)$, buildHeap $O(n)$

힙의 배열 표현 (1-index)

- 노드 i 의 자식은 $A[2i]$ 와 $A[2i+1]$ 이다.
- 노드 i 의 부모는 $A[\lfloor i/2 \rfloor]$ 이다.
- **최대 힙(Max-Heap)** 에서는 모든 부모 키가 자식 키 이상이다.

정렬 완전 비교표

알고리즘	최악	평균	최선	공간	안정성	제자리
선택	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	×	✓
버블	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$O(1)$	✓	✓
삽입	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$O(1)$	✓	✓
병합	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n\log n)$	$O(n)$	✓	×
퀵	$\Theta(n^2)$	$\Theta(n\log n)$	$\Theta(n\log n)$	$O(\log n)$	×	✓
힙	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n\log n)$	$O(1)$	×	✓
계수	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$O(k)$	✓	×
기수	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$O(n+k)$	✓	×

비교 기반 하한

- 모든 비교 기반 정렬은 최악 $\Omega(n\log n)$ 비교가 필요하다.
- 결정 트리 모델로 증명된다: $n!$ 개의 순열을 구별하려면 높이가 $\log_2(n!) = \Theta(n\log n)$ 이상이어야 한다.
- 계수 정렬과 기수 정렬은 비교를 쓰지 않으므로 이 하한을 우회할 수 있다.

정렬 선택 가이드

- 안정성이 필수이면 병합 정렬을 쓴다.
- 메모리 제약이 있고 최악 성능 보장이 필요하면 힙 정렬을 쓴다.
- 일반적으로 가장 빠른 정렬이 필요하면 랜덤 피벗 퀵 정렬을 쓴다.
- 작거나 거의 정렬된 입력에는 삽입 정렬이 유리하다.
- 정수 값의 범위가 제한적이면 계수 정렬 또는 기수 정렬을 쓴다.

기수 정렬의 LSD 순서

- 최하위(LSD) 자리부터 최상위(MSD) 자리까지 순서대로 정렬한다.
- 각 자리 정렬은 반드시 안정 정렬(보통 계수 정렬)이어야 한다.
- 이전 자리의 순서가 유지되어야 최종적으로 모든 자리가 정렬된다.

4주차 분할 정복

기본 패턴

```
DivideAndConquer(P):
    if P 가 충분히 작으면:
        직접 해결
    else:
        DIVIDE:   P 를 부분 문제 P1, ..., Pk 로 분할
        CONQUER:  각 Pi 를 재귀로 해결
        COMBINE:  부분 해들을 원래 문제의 해로 결합
```

설명 — 큰 문제를 독립적인 작은 부분 문제로 나누어 재귀로 풀고 결합하는 패러다임이다. 부분 문제가 서로 겹치지 않으므로 각 부분 문제를 한 번씩만 풀면 된다. 점화식은 보통 $T(n) = aT(n/b) + f(n)$ 의 형태를 가지며 마스터 정리로 해결한다. 부분 문제가 겹치면 동적 계획법을 써야 한다.

대표 알고리즘

알고리즘	점화식	복잡도
병합 정렬	$2T(n/2) + \Theta(n)$	$\Theta(n\log n)$
이진 탐색	$T(n/2) + O(1)$	$\Theta(\log n)$
퀵 정렬 (평균)	$2T(n/2) + \Theta(n)$	$\Theta(n\log n)$
퀵 정렬 (최악)	$T(n-1) + \Theta(n)$	$\Theta(n^2)$
랜덤 선택	$T(3n/4) + \Theta(n)$	평균 $\Theta(n)$
중앙값의 중앙값	$T(n/5) + T(7n/10) + \Theta(n)$	최악 $\Theta(n)$
최근접 쌍	$2T(n/2) + O(n\log n)$	$O(n\log^2 n)$

랜덤 선택 (퀵 선택)

```
select(A, p, r, i):
    if p == r: return A[p]
    q = partition(A, p, r)
    k = q - p + 1
    if i == k:  return A[q]
    elif i < k: return select(A, p, q-1, i)
    else:       return select(A, q+1, r, i - k)
```

설명 — 퀵 정렬의 파티션을 재사용하여 피벗의 순위 k 를 계산한 뒤, 원하는 i 가 어느 쪽에 있는지 판단해 한쪽 부분 배열만 재귀 호출한다. 양쪽을 모두 정렬할 필요가 없기 때문에 평균 시간은 $\Theta(n)$ 이다. 오른쪽으로 갈 때는 피벗과 왼쪽 부분을 건너뛴 만큼 i 를 $i - k$ 로 보정한다.

중앙값의 중앙값 (Median of Medians)

- 원소를 5개씩 묶은 뒤 각 그룹의 중앙값을 구하고, 그 중앙값들의 중앙값 M 을 재귀로 찾는다.
- 이 M 을 피벗으로 쓰면 최소 $3n / 10$ 개 원소가 M 이하이고 최소 $3n / 10$ 개가 M 이상이므로, 재귀 크기는 최대 $7n / 10$ 이 된다.
- 점화식 $T(n) = T(n / 5) + T(7n / 10) + \Theta(n)$ 은 $1 / 5 + 7 / 10 < 1$ 이므로 $\Theta(n)$ 으로 수렴한다.

최근접 쌍

1. 점들을 x 좌표 기준으로 정렬한다.
2. 중앙 x 로 좌/우를 분할하여 각각 재귀 호출한다.
3. $d = \min(d_L, d_R)$ 을 구한다.
4. 분할선에서 $\pm d$ 이내의 점들로 스트립을 구성하고 y 좌표로 정렬한다.
5. 스트립 안에서 각 점은 y 순서상 최대 7개 이웃만 검사하면 충분하다.

퀵 정렬의 Partition (Lomuto)

```
partition(A, p, r):
    pivot = A[r]
    i = p - 1
    for j = p to r - 1:
        if A[j] ≤ pivot:
            i += 1
            swap A[i] ⇔ A[j]
    swap A[i+1] ⇔ A[r]
    return i + 1
```

설명 — 배열의 마지막 원소를 피벗으로 쓴다. 포인터 `i` 는 "피벗 이하 원소들의 끝" 을 가리키고 `j` 는 현재 검사 중인 위치이다. `A[j] ≤ pivot` 이면 `i` 를 증가시키고 스왑해 이하 영역을 확장한다. 루프가 끝난 뒤 피벗을 $i + 1$ 위치로 옮겨 최종 정렬 위치를 확정하고 그 인덱스를 반환한다. 반환 인덱스 왼쪽은 이하, 오른쪽은 초과가 된다.

최악을 피하는 방법

- **랜덤 피벗** — 무작위 인덱스를 선택해 마지막과 스왑한 뒤 파티션한다.
- **Median-of-3** — 첫 원소, 중간 원소, 마지막 원소의 중앙값을 피벗으로 쓴다.
- **Introsort** — 재귀 깊이가 일정 수준을 넘으면 힙 정렬로 전환한다.

분할 정복이 실패하는 경우

- 순진한 재귀 피보나치는 $T(n) = T(n-1) + T(n-2) + 1 \rightarrow O(\phi^n)$ 로 지수 시간이다.
- 원인은 같은 부분 문제가 여러 번 반복 계산되는 중복 부분 문제(Overlapping Subproblems)에 있다.
- 이러한 문제는 DP로 해결한다.

5주차 그리디

패러다임

매 단계에서 지금 당장 가장 좋아 보이는 선택(지역 최적)을 하고, 이미 내린 선택을 되돌리지 않는다.

정확성을 보장하는 두 조건

1. **그리디 선택 속성(Greedy-Choice Property)** — 어떤 최적해는 반드시 그리디 선택을 포함한다. 보통 교환 논증(Exchange Argument)으로 증명한다.
2. **최적 부분 구조(Optimal Substructure)** — 그리디 선택 후 남은 부분 문제의 최적해가 전체 최적해의 일부가 된다.

그리디가 실패하는 반례

문제	그리디 결과	최적 결과
동전 {1, 3, 4}, 금액 6	4 + 1 + 1 = 3개	3 + 3 = 2개
동전 {1, 5, 7, 10}, 금액 14	10 + 1 + 1 + 1 + 1 = 5개	7 + 7 = 2개
0/1 배낭 (가치/무게 정렬)	용량이 남아도 분수 사용 불가	DP로 풀어야 한다

대표 알고리즘

문제	그리디 전략	시간 복잡도
동전 거스름돈 (표준 화폐)	큰 단위부터 선택	$O(k)$
분할 가능 배낭	가치/무게 비율 내림차순	$O(n\log n)$
활동 선택 (최대 비겹침)	가장 빨리 끝나는 활동(EFT)	$O(n\log n)$
작업 스케줄링 (최소 기계)	가장 빨리 시작하는 작업(EST)	$O(n\log n)$

허프만 코딩	최저 빈도 두 노드를 병합	$O(n\log n)$
크루스칼 MST	가중치 오름차순 간선 + Union-Find	$O(m\log m)$
프림 MST	시작 정점에서 트리를 키움	$O(n^2)$ 또는 $O(m\log n)$
다익스트라	최소 거리 미확정 정점 확정	$O(n^2)$ 또는 $O(m\log n)$

활동 선택 vs 작업 스케줄링 (혼동 주의)

구분	활동 선택	작업 스케줄링
목표	활동 수 최대화	기계 수 최소화
그리디 기준	EFT (가장 빠른 종료 시간)	EST (가장 빠른 시작 시간)
기계 수	1개	여러 개

허프만 코딩

```
1. 문자 빈도로 최소 힙 Q 를 구성한다.
2. while |Q| ≥ 2:
     A = extract-min(Q)
     B = extract-min(Q)
     N = 새 노드, N.freq = A.freq + B.freq
     N.left = A,  N.right = B
     insert(Q, N)
3. 루트에서 리프까지의 0/1 경로가 각 문자의 접두 코드이다.
```

설명 — 빈도가 가장 낮은 두 노드를 뽑아 합친 뒤 새 노드로 힙에 다시 넣는 과정을 반복한다. 합쳐지는 노드는 트리에서 더 깊게 위치하므로 긴 코드가 배정되고, 빈도가 높은 노드는 얕게 위치해 짧은 코드가 배정된다. 루트에서 각 리프까지의 0/1 경로가 바로 접두 코드가 된다. 최적성은 교환 논증으로 증명되며, 핵심은 최저 빈도 두 문자가 반드시 트리에서 형제라는 점이다.

크루스칼 vs 프림

구분	크루스칼	프림
접근	간선 중심	정점 중심
자료구조	Union-Find	우선순위 큐
복잡도	$O(m\log m)$	$O(n^2)$ 또는 $O(m\log n)$
적합 그래프	희소 그래프	밀집 그래프

다익스트라

```
D[v] = ∞ ; D[s] = 0 ; S = {}
while S ≠ V:
    u = S 밖에서 D[u] 가 최소인 정점
    S += {u}
    for v in neighbors(u) \ S:
        if D[u] + w(u, v) < D[v]:
            D[v] = D[u] + w(u, v)     # 간선 완화
```

설명 — 출발 정점 s 에서 각 정점까지의 최단 거리 $D[v]$ 를 계산한다. 매 반복에서 아직 확정되지 않은 정점 중 D 가 최소인 u 를 확정 집합 S 에 추가하고, u 의 이웃들에 대해 간선 완화를 수행한다. 음이 아닌 가중치 가정 덕분에 S 에 편입된 정점의 D 값은 이후 바뀌지 않는다는 불변식이 성립한다.

음수 간선 주의 — 음수 간선이 있으면 이미 확정된 $D[u]$ 가 나중에 더 작아질 수 있어 그리디 불변식이 깨진다. 이 경우 벨만-포드 알고리즘($O(VE)$)을 써야 한다.

6주차 동적 계획법

DP 의 두 필수 조건

1. **최적 부분 구조(Optimal Substructure)** — 전체 최적해가 부분 문제 최적해를 포함한다.
2. **중복 부분 문제(Overlapping Subproblems)** — 같은 부분 문제가 재귀 구조에서 여러 번 등장한다.

메모이제이션 vs 테이블화

구분	메모이제이션 (하향식)	테이블화 (상향식)
구현	재귀와 캐시 결합	반복문으로 테이블을 채움
계산 범위	실제 필요한 부분 문제만	모든 부분 문제
오버헤드	재귀 호출 스택	타이트 루프
실전 속도	상대적으로 느리다	상대적으로 빠르다

DP 6단계 레시피

1. 부분 문제를 명확히 정의한다 (예: `dp[i, j]` 가 무엇을 뜻하는가).
2. 점화식을 작성한다.

3. 기저 조건을 식별한다.
4. 부분 문제를 채우는 순서를 결정한다.
5. 최종 답이 어느 셀에 저장되는지 확인한다.
6. 필요하면 역추적으로 실제 해를 복원한다.

대표 DP 문제 요약

문제	부분 문제	점화식 요약	시간	공간
피보나치	$f[i]$	$f[i-1]+f[i-2]$	$O(n)$	$O(1)$
행렬 경로	$c[i,j]$	$m[i,j]+\max(c[i-1,j],c[i,j-1])$	$O(n^2)$	$O(n^2)$
LCS	$c[i,j]$	일치 시 $+1$, 불일치 시 $\max$	$O(mn)$	$O(mn)$
편집 거리	$E[i,j]$	일치 시 대각선, 불일치 시 $1+\min$	$O(mn)$	$O(mn)$
0/1 배낭	$K[i,w]$	$\max(K[i-1,w],v_i+K[i-1,w-w_i])$	$O(nC)$	$O(nC)$
동전 거스름돈 (최소)	$C[j]$	$1+\min_{d_i\le j}C[j-d_i]$	$O(nk)$	$O(n)$
플로이드-워셜	$d[i,j,k]$	$\min(d[i,j],d[i,k]+d[k,j])$	$O(n^3)$	$O(n^2)$

LCS 점화식

```
if X[i] == Y[j]: c[i][j] = c[i-1][j-1] + 1
else:            c[i][j] = max(c[i-1][j], c[i][j-1])
```

설명 — $c[i][j]$ 는 $X[1..i]$ 와 $Y[1..j]$ 의 LCS 길이이다. 마지막 문자가 일치하면 두 문자를 모두 제거한 부분 수열의 LCS 에 1 을 더하고, 불일치하면 X 의 마지막을 뺀 경우와 Y 의 마지막을 뺀 경우 중 더 긴 쪽을 선택한다. 실제 LCS 문자열은 역추적으로 복원한다.

편집 거리 점화식

```
if X[i] == Y[j]: E[i][j] = E[i-1][j-1]
else:            E[i][j] = 1 + min(E[i-1][j],       # 삭제
                                   E[i][j-1],       # 삽입
                                   E[i-1][j-1])     # 교체
```

설명 — $E[i][j]$ 는 $X[1..i]$ 를 $Y[1..j]$ 로 바꾸는 최소 연산 수이다. 마지막 문자가 일치하면 대각선 값을 그대로 취하고(연산 0), 불일치하면 삭제·삽입·교체의 세 경로 중 최소에 1 을 더한다. 기저 조건은 $E[i][0] = i$ (i 번 삭제), $E[0][j] = j$ (j 번 삽입)이다.

0/1 배낭 점화식

```
if w[i] > w:  K[i][w] = K[i-1][w]
else:         K[i][w] = max(K[i-1][w],
                            v[i] + K[i-1][w - w[i]])
```

설명 — $K[i][w]$ 는 물건 1..i 까지 고려하고 용량이 w 일 때의 최대 가치이다. 각 물건을 넣을지 말지 두 선택을 비교한다. 넣으려면 용량이 w_i 이상이어야 하고, 넣을 경우 가치는 "나머지 용량으로 $i-1$ 까지 담은 최대 가치 $+ v_i$" 가 된다. 넣지 않으면 $i-1$ 까지 담은 값을 그대로 가져간다. 둘 중 큰 쪽을 선택한다.

플로이드-워셜

```
for k = 1 to n:     # k 가 반드시 가장 바깥 루프
    for i = 1 to n:
        for j = 1 to n:
            D[i][j] = min(D[i][j], D[i][k] + D[k][j])
```

설명 — 중간 경유 정점 후보를 $1 \to 1,2 \to \cdots \to 1..n$ 으로 점차 확장하며 모든 쌍 $i \to j$ 의 최단 거리를 갱신한다. 핵심 아이디어는 "정점 k 를 경유하면 더 가까워지는가?" 를 모든 (i, j) 에 대해 매 단계 확인하는 것이다. k 가 반드시 최외곽 루프여야 하는 이유는, 각 k 의 갱신이 "${1..k-1}$ 만 경유할 때의 결과" 를 바탕으로 이루어져야 하기 때문이다. 음수 간선은 허용되지만 음의 사이클은 없어야 한다. 복잡도는 $\Theta(n^3)$ 이다.

공식 치트시트

점화식 빠른 참조

점화식	해	대표 예시
$T(n) = T(n/2) + 1$	$\Theta(\log n)$	이진 탐색
$T(n) = T(n-1) + 1$	$\Theta(n)$	팩토리얼
$T(n) = 2T(n/2) + n$	$\Theta(n\log n)$	병합 정렬

$T(n) = 2T(n/2) + 1$	$\Theta(n)$	—
$T(n) = T(n-1) + n$	$\Theta(n^2)$	삽입 정렬, 퀵 정렬 최악
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	하노이 탑
$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(\phi^n)$	순진한 재귀 피보나치

Big-O 증명 공식

다항식 $f(n) = \sum a_i n^i$ 에 대해 $c = \sum |a_i|$, $n_0 = 1$ 로 잡으면 대부분 성립한다.

유클리드 GCD

$$\gcd(a, b) = \gcd(b, a \bmod b), \ \gcd(a, 0) = a$$

비교 기반 정렬 하한

모든 비교 기반 정렬은 최악 $\Omega(n\log n)$ 비교가 필요하며, 결정 트리 모델로 증명된다.

패러다임 결정 플로우

```
                 문제 읽기
                    |
                    ▼
        제약이 작은가? (n ≤ 수백)
                 /  \
              예      아니오
              |        |
              ▼        ▼
       완전 탐색    독립 부분 문제로 분할 가능?
                       /   \
                    예      아니오
                     |        |
                     ▼        ▼
                 분할 정복    지역 최적이 전역 최적?
                              /   \
                           예      아니오
                            |        |
                            ▼        ▼
                         그리디    중복 부분 문제 +
                                   최적 부분 구조?
                                        |
                                        ▼
                                       DP
```

설명 — 새로운 문제를 받으면 위에서부터 질문에 따라 내려간다. (1) 제약이 작으면 $O(n^2)$ 도 허용되므로 완전 탐색을 쓴다. (2) 큰 입력이면 부분 문제가 독립적인지 확인하고, 그렇다면 분할 정복을 쓴다. (3) 그렇지 않으면 그리디 선택이 항상 안전한지(교환 논증으로 증명 가능한지) 확인한다. (4) 마지막으로 중복 부분 문제와 최적 부분 구조가 있다면 DP 를 쓴다. 실전에서는 그리디를 먼저 시도하고 반례가 보이면 DP 로 전환하는 것이 경험칙이다.

패러다임 비교

측면	분할 정복	그리디	동적 계획법
부분 문제	독립적	해당 없음	중복
방향	재귀	순차	상향식 또는 메모이제이션
최적 보장	문제에 따라 다름	그리디 속성 필요	항상
전형 복잡도	$O(n\log n)$	$O(n\log n)$ 또는 $O(n)$	$O(n^2)$ 또는 $O(n^3)$
대표 예시	병합·퀵·이진 탐색	Dijkstra·Huffman·MST	LCS·편집 거리·배낭·FW

같은 문제, 다른 패러다임

문제	그리디	DP	분할 정복
동전 (표준 화폐)	✓	✓	—
동전 (임의 화폐)	×	✓	—
분할 가능 배낭	✓	✓	—
0/1 배낭	×	✓	—
최단 경로 (가중치 ≥ 0)	✓ (Dijkstra)	✓	—
최단 경로 (음수 허용)	—	✓ (FW, 벨만-포드)	—
정렬	—	—	✓
k번째 원소	—	—	✓
MST	✓	—	—

시험 문제 스타일 예시

예시 1 — 마스터 정리

$T(n) = 4T(n/2) + n$ 의 복잡도를 구한다.

- $a = 4$, $b = 2$ 이므로 $n^{\log_b a} = n^2$ 이다.
- $f(n) = n = O(n^{2-1})$ 이므로 Case 1 에 해당한다.
- 따라서 $T(n) = \Theta(n^2)$ 이다.

예시 2 — 그리디 vs DP

동전 {1, 3, 4}, 금액 6 을 비교한다.

- **그리디** — 4 + 1 + 1 로 3개가 된다.
- **DP** — $C[6] = 1 + \min(C[5], C[3], C[2]) = 1 + \min(2, 1, 2) = 2$ 이므로 3 + 3 으로 2개가 된다.
- 이 예시에서 그리디는 최적이 아니다.

예시 3 — LCS 테이블

X = "ABCB", Y = "BDCAB" 의 LCS 를 구한다.

	""	B	D	C	A	B
""	0	0	0	0	0	0
A	0	0	0	0	**1**	1
B	0	**1**	1	1	1	**2**
C	0	1	1	**2**	2	2
B	0	1	1	2	2	**3**

답은 길이 3, LCS 의 예는 "BCB" 이다.

예상 시험문제와 모범 답안 10선

시험에서 같은 문제가 나올 수도 있고, 유사 문제에 답안 구조를 그대로 이식할 수도 있다. 각 답안은 "정의 — 왜 필요한가 — 메커니즘 — 예시·증명 — 한계·연결" 패턴을 따른다.

Q1. Big-O 증명 — $2n^2 + 3n + 5 = O(n^2)$ 임을 증명하라.

답안 — Big-O 의 정의에 따라 상수 c 와 n_0 을 제시하면 증명이 완성된다. $c = 10$, $n_0 = 1$ 로 두면 모든 $n \geq 1$ 에 대해 $2n^2 + 3n + 5 \leq 2n^2 + 3n^2 + 5n^2 = 10n^2 = c \cdot n^2$ 이 성립한다. 따라서 정의에 의해 $2n^2 + 3n + 5 = O(n^2)$ 이다. 이는 "f 가 g 보다 점근적으로 빨리 자라지 않는다" 는 의미이며, 상수 배율을 무시한 상한이라는 점이 핵심이다. 참고로 이 식은 $\Theta(n^2)$ 이기도 하므로 실제 답안에서는 가능한 한 더 타이트한 Θ 표기를 쓰는 것이 정보력이 높다.

Q2. 마스터 정리 — $T(n) = 3T(n/2) + n^2$ 의 시간 복잡도를 구하라.

답안 — 마스터 정리의 표준 형태 $T(n) = aT(n/b) + f(n)$ 에서 $a = 3$, $b = 2$ 이므로 비교 함수는 $h(n) = n^{\log_2 3} \approx n^{1.585}$ 이다. $f(n) = n^2$ 을 $h(n)$ 과 비교하면 $n^2 = \Omega(n^{1.585+\varepsilon})$ 이 $\varepsilon \approx 0.4$ 로 성립한다. 정규성 조건(Regularity Condition) $af(n/b) \leq cf(n)$ 은 $3(n/2)^2 = 0.75n^2 \leq 0.75n^2$ 로 $c = 0.75 < 1$ 에 대해 성립하므로 Case 3 에 해당한다. 따라서 $T(n) = \Theta(f(n)) = \Theta(n^2)$ 이다. 이 경우 루트에서의 작업량이 재귀 비용을 지배한다는 의미이며, 이는 재귀 트리의 각 레벨 비용이 기하급수적으로 감소하는 구조이다.

Q3. 비교 기반 정렬의 하한 — 모든 비교 기반 정렬이 최악 $\Omega(n\log n)$ 비교가 필요한 이유를 결정 트리 모델로 설명하라.

답안 — 임의의 비교 기반 정렬은 결정 트리(Decision Tree)로 모델링할 수 있으며, 각 내부 노드는 "$A[i] \leq A[j]$?" 형태의 한 번의 비교를, 각 리프는 가능한 출력 순열 하나에 대응한다. n 개 원소의 순열은 $n!$ 개이므로 결정 트리는 적어도 $n!$ 개의 리프를 가져야 한다. 이진 트리의 높이 h 에 대해 리프 수는 최대 2^h 이므로 $2^h \geq n!$ 이다. 양변에 로그를 취하면 $h \geq \log_2(n!) = \Theta(n\log n)$ (Stirling 근사에 의해) 이 되며, 트리의 높이가 곧 최악 비교 횟수이므로 $\Omega(n\log n)$ 이 하한이다. 이 하한은 **비교라는 모델 자체의 한계** 이며, 계수 정렬(Counting Sort)이나 기수 정렬(Radix Sort)이 값의 범위나 자릿수 정보를 활용해 $O(n)$ 을 달성할 수 있는 이유도 같은 논리로 설명된다.

Q4. 퀵 정렬 최악 케이스 — 퀵 정렬이 언제 $\Theta(n^2)$ 이 되며, 이를 어떻게 회피하는지 설명하라.

답안 — 퀵 정렬(Quick Sort)의 각 호출은 피벗(Pivot) 기준 파티션 뒤 두 영역을 재귀로 처리한다. 최악은 파티션이 $0 : n-1$ 로 완전히 불균형할 때 발생하며, 이때 점화식은 $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$ 이 된다. 대표적 최악 입력은 **이미 정렬된 배열에 마지막 원소를 피벗으로 쓰는 Lomuto 분할** 이다. 회피 전략은 세 가지이다. (1) **랜덤 피벗** 은 임의 인덱스를 골라 마지막과 스왑하여 최악 기대 확률을 낮추며, 평균 $\Theta(n\log n)$ 을 얻는다. (2) **Median-of-3** 는 첫·중간·마지막 세 원소의 중앙값을 피벗으로 써 이미 정렬된 입력도 안전하게 처리한다. (3) **Introsort** 는 재귀 깊이가 $2\log n$ 을 넘으면 힙 정렬로 전환하여 최악 $O(n\log n)$ 을 보장하며, C++ STL 의 `std::sort` 가 이 방식을 쓴다. 트레이드오프로 퀵 정렬은 제자리 정렬이자 캐시 친화적이어서 평균적으로 병합 정렬보다 빠르지만, 최악 위험을 반드시 관리해야 한다.

Q5. buildHeap 이 $O(n)$ 인 이유 — 모든 노드에 heapify 를 적용하는데도 왜 $O(n\log n)$ 이 아닌지 설명하라.

답안 — 직관적으로는 n 개 노드 각각에 $O(\log n)$ 의 heapify 를 적용하므로 $O(n\log n)$ 처럼 보이지만, **리프 노드는 heapify 가 필요 없다** 는 관찰이 타이트 분석의 핵심이다. 완전 이진 트리(Complete Binary Tree)에서 높이 h 인 노드는 약 $n / 2^{h+1}$ 개이며, 각 노드에 대한 heapify 비용은 $O(h)$ 이다. 총 비용은 다음과 같다. $$ \sum_{h=0}^{\log n} \frac{n}{2^{h+1}} \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = O(n \cdot 2) = O(n) $$ 무한 급수 $\sum h / 2^h = 2$ 이기 때문이다. 즉 낮은 높이(리프에 가까운)의 노드가 대부분을 차지하고 이들의 작업량은 거의 상수이기에, 깊이별 가중합이 선형으로 수렴한다. 이 결과는 힙 정렬의 전체 복잡도를 $O(n) + O(n\log n) = O(n\log n)$ 으로 정리하는 데 쓰인다.

Q6. 활동 선택의 그리디 최적성 증명 — 가장 빨리 끝나는 활동(EFT)부터 선택하는 그리디가 최적임을 증명하라.

답안 — 교환 논증(Exchange Argument)으로 증명한다. 그리디 해를 $G = g_1, g_2, ..., g_k$, 임의의 최적해를 $O = o_1, o_2, ..., o_m$ 이라 두고 모두 종료 시간 오름차순으로 정렬되어 있다고 하자. 그리디는 **전체 활동 중 가장 일찍 끝나는** 활동을 첫 번째로 선택하므로 $\text{finish}(g_1) \leq \text{finish}(o_1)$ 이 성립한다. 따라서 O 에서 o_1 을 g_1 로 교환해도 이후 활동들은 여전히 겹치지 않으며 크기도 변하지 않는다. 즉 $O^{'} = g_1, o_2, ..., o_m$ 도 최적해이다. 이 논리를 귀납적으로 적용하면 O 의 모든 원소를 그리디 선택으로 교체해도 최적성이 유지되므로 $|G| = |O|$ 이다. 핵심 직관은 "가장 빨리 끝나는 활동을 택해야 이후 활동을 위한 시간이 최대로 남는다" 는 것이며, 이로써 그리디 선택 속성(Greedy-Choice Property)과 최적 부분 구조(Optimal Substructure)가 모두 확보된다. 복잡도는 종료 시간 정렬이 지배해 $O(n\text{log}n)$ 이다.

Q7. 다익스트라 vs 벨만-포드 — 두 최단 경로 알고리즘의 차이와 언제 어느 것을 써야 하는지 설명하라.

답안 — 다익스트라(Dijkstra) 알고리즘은 음이 아닌 가중치 그래프에서 단일 출발점 최단 경로를 $O(m\text{log}n)$ (이진 힙 기반) 에 구하는 그리디 알고리즘이다. 매 단계 미확정 정점 중 $D[u]$ 가 최소인 u 를 확정하고 이웃의 거리를 완화(Edge Relaxation)하는 구조이며, 그리디 불변식은 "u 가 확정되면 $D[u]$ 는 이후 절대 바뀌지 않는다" 이다. 음수 간선이 있으면 나중에 더 짧은 경로가 발견될 수 있어 이 불변식이 깨지므로 잘못된 답을 낸다. 벨만-포드(Bellman-Ford)는 모든 간선을 $V - 1$ 번 완화하는 DP 계열 알고리즘으로 시간은 $O(VE)$ 이지만 **음수 간선을 허용** 하고 V 번째 반복에서 거리가 또 갱신되면 음의 사이클(Negative Cycle)을 검출한다. 정리하면, 가중치가 모두 음이 아니고 속도가 중요하면 다익스트라, 음수 간선이 있거나 음의 사이클을 탐지해야 하면 벨만-포드를 쓴다. 모든 쌍의 최단 경로가 필요하면 플로이드-워셜($O(n^3)$) 을 선택한다.

Q8. 0/1 배낭 — 물건 $(w, v) = (2, 3), (3, 4), (4, 5), (5, 6)$ 과 용량 $C = 5$ 에 대해 DP 로 최대 가치를 구하고, 왜 그리디가 실패하는지 설명하라.

답안 — 부분 문제를 $K[i][w]$ = "물건 $1..i$ 까지를 고려하고 남은 용량이 w 일 때 최대 가치" 로 정의하면 점화식은 $K[i][w] = \max(K[i-1][w], ; v_i + K[i-1][w - w_i])$ 이다. 기저는 $K[0][w] = K[i][0] = 0$ 이다. 테이블은 다음과 같다.

$i \backslash w$	**0**	**1**	**2**	**3**	**4**	**5**
0	0	0	0	0	0	0
1 (2,3)	0	0	3	3	3	3
2 (3,4)	0	0	3	4	4	**7**
3 (4,5)	0	0	3	4	5	7
4 (5,6)	0	0	3	4	5	7

최대 가치는 $K[4][5] = 7$ 이며, 선택된 물건은 1번과 2번(무게 $2 + 3 = 5$, 가치 $3 + 4 = 7$)이다. 그리디(가치/무게 비율 큰 순: 1번 1.5, 2번 1.33, 3번 1.25, 4번 1.2)로 풀면 1번(2, 3)을 담은 뒤 남은 용량 3에 2번(3, 4)을 담을 수는 있어 이 특정 예는 그리디도 7을 얻는다. 그러나 일반적으로 0/1 배낭에서는 비율이 큰 물건을 넣었더니 남은 용량이 다음 물건에 부족해 v가 더 큰 조합을 놓치는 경우가 생기며(예: 용량 5, 물건 (4, 8), (3, 5), (2, 4) 이면 그리디는 8, DP 는 9 이다), 이는 물건을 쪼갤 수 없기 때문이다. 분할 가능 배낭(Fractional Knapsack)은 물건을 쪼갤 수 있어 그리디가 항상 최적이지만, 0/1 배낭은 DP 로만 최적을 보장한다. 복잡도는 $O(nC)$ 이며 이는 의사다항시간(Pseudopolynomial Time)이다.

Q9. LCS 설계 — X = "AGCAT", Y = "GAC" 의 LCS 를 DP 로 구하고 점화식의 각 경우가 왜 타당한지 설명하라.

답안 — 부분 문제를 $c[i][j]$ = "$X[1..i]$ 와 $Y[1..j]$ 의 LCS 길이" 로 정의한다. 점화식은 다음과 같다. $$ c[i][j] = \begin{cases} 0 & i=0 \text{ 또는 } j=0 \ c[i-1][j-1] + 1 & X[i] = Y[j] \ \max(c[i-1][j],; c[i][j-1]) & X[i] \neq Y[j] \end{cases} $$ 각 분기의 정당성은 다음과 같다. (1) 문자가 **일치** 하면 그 문자를 LCS 에 포함시키는 것이 항상 최적이다. 두 문자열의 마지막 문자를 쓰면 남은 문제가 $X[1..i-1]$ 과 $Y[1..j-1]$ 의 LCS 에 1을 더하는 구조가 된다. (2) 문자가 **불일치** 하면 $X[i]$ 또는 $Y[j]$ 중 하나는 LCS 에 포함되지 않으므로, 두 가지 경우 $c[i-1][j]$ 와 $c[i][j-1]$ 중 큰 값을 택한다. 테이블은 다음과 같다.

	""	G	A	C
""	0	0	0	0
A	0	0	1	1
G	0	1	1	1
C	0	1	1	**2**
A	0	1	2	2
T	0	1	**2**	2

$c[5][3] = 2$ 가 답이며 LCS 의 예는 "GA" 또는 "AC" 이다. 역추적은 $c[m][n]$ 에서 출발해 문자 일치 시 대각선으로 이동하며 문자를 기록하고, 아니면 $c[i-1][j]$ 와 $c[i][j-1]$ 중 큰 쪽으로 이동한다. 복잡도는 시간·공간 모두 $O(mn)$ 이다. LCS 는 Git diff, DNA 서열 정렬, 편집 거리의 변형 등에 쓰인다.

Q10. 루프 복잡도 분석 — 다음 함수의 시간 복잡도를 구하고 근거를 설명하라.

```python
def f(n):
    count = 0
    i = 1
    while i < n:
        for j in range(n):
            count += 1
        i *= 2
    return count
```

답안 — 핵심은 두 루프의 증가 패턴을 구분하는 것이다. 바깥 `while` 은 `i *= 2` 로 i 가 2 배씩 증가하므로 조건 $i < n$ 이 $i \geq n$ 으로 바뀔 때까지 대략 $\lfloor \log_2 n \rfloor$ 회 반복된다. 안쪽 `for` 는 j 를 0 부터 $n-1$ 까지 1씩 증가시키므로 매번 정확히 n 회 실행된다. 따라서 총 연산 횟수는 $n \cdot \log_2 n = \Theta(n \log n)$ 이다. 일반 원칙은 `i *= 2` 또는 `i /= 2` 처럼 **곱셈·나눗셈으로 변하는 루프는** $O(\log n)$, `i += 1` 처럼 **덧셈으로 변하는 루프는** $O(n)$ 이며, 중첩되면 두 수를 곱한 것이 전체 복잡도가 된다. 참고로 $n = 16$ 을 대입하면 바깥 반복은 $i = 1, 2, 4, 8$ 로 4회, 안쪽은 매번 16회 실행되므로 `count = 64` 가 되어 $n \log_2 n = 16 \cdot 4 = 64$ 와 일치한다.

자주 하는 실수

실수	회피 방법
O, Ω, Θ 를 혼동한다	O 는 상한, Ω 은 하한, Θ 는 양쪽이 모두 꼭 맞을 때이다.
비다항식 $f(n)$ 에 마스터 정리를 적용한다	정규성 조건을 확인하고, 안 되면 반복 대입이나 귀납법을 쓴다.
증명 없이 그리디를 최적이라 주장한다	반드시 반례를 먼저 찾아보고, 없으면 교환 논증으로 증명한다.
DP 기저 조건을 잘못 잡는다	$n = 1, 2, 3$ 을 손으로 추적해 검증한다.
DP 테이블 인덱스를 헷갈린다	0-index 와 1-index 중 무엇을 쓰는지 명시한다.
분할 가능 배낭과 0/1 배낭을 혼동한다	분할 가능은 그리디, 0/1 은 DP 이다.
퀵 정렬의 n^2 최악을 간과한다	랜덤 피벗이나 Median-of-3 를 사용해야 한다.
플로이드-워셜의 k 루프 순서를 바꾼다	k 는 반드시 가장 바깥 루프여야 한다.
다익스트라에 음수 간선을 넣는다	음수 간선이 있으면 벨만-포드를 써야 한다.

귀납에서 $cn + 1 \leq cn$ 으로 막힌다	추측을 $cn - d$ 형태로 강화해 여유를 만든다.
병합 정렬의 공간 $O(n)$ 을 빠뜨린다	보조 배열이 필요함을 반드시 명시한다.
이진 탐색의 `left <= right` 등호를 뺀다	등호가 없으면 마지막 한 원소를 검사하지 못한다.
buildHeap 을 $O(n\log n)$ 이라 쓴다	실제로는 $O(n)$ 이며, 리프 절반을 처리하지 않는 타이트 분석 덕분이다.
계수 정렬에서 순방향 순회로 안정성을 잃는다	반드시 역순 순회로 배치해야 안정성이 보장된다.
LCS 를 부분 문자열과 혼동한다	LCS 는 연속하지 않아도 되는 부분 수열이다.

마지막 갱신 — 2026-04-23